

Docker Project Doc

Documented By: NISHAT AFLA

The work I did in this project:

1. Containerized full-stack applications using Docker (ReactJS frontend with NGINX, Laravel backend with PHP-Apache).
2. Created custom Dockerfiles
3. Built, tagged, and managed Docker images and containers on Ubuntu.
4. Used Docker Compose to orchestrate multi-container applications.
5. Created snapshot images and managed container lifecycle operations.
6. Pushed versioned images to Docker Hub repositories.
7. Deployed containers on a GCP VM instance.
8. Configured port mappings and firewall rules for external access.
9. Troubleshot issues like 403/500 errors, migration errors, and container configuration problems.
10. Gained practical experience in Linux, Docker, cloud deployment, and debugging.

Docker Installation Steps

****Update system:****

`sudo apt update && sudo apt upgrade -y`

```
sudo apt install docker.io # version 26.1.3-0ubuntu1~24.04.1
sudo apt install podman-docker # version 4.9.3+ds1-1ubuntu0.2
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for nishat:
```

****Install dependencies:****

`sudo apt install ca-certificates curl gnupg lsb-release`

```
Processing triggers for libc-bin (2.39-0ubuntu8.4) ...
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ sudo apt install ca-certificates curl gnupg lsb-release
[sudo] password for nishat:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

****Add Docker GPG key:****

`sudo mkdir -p /etc/apt/keyrings`

`curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \`

`sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg`

****Add Docker's repository:****

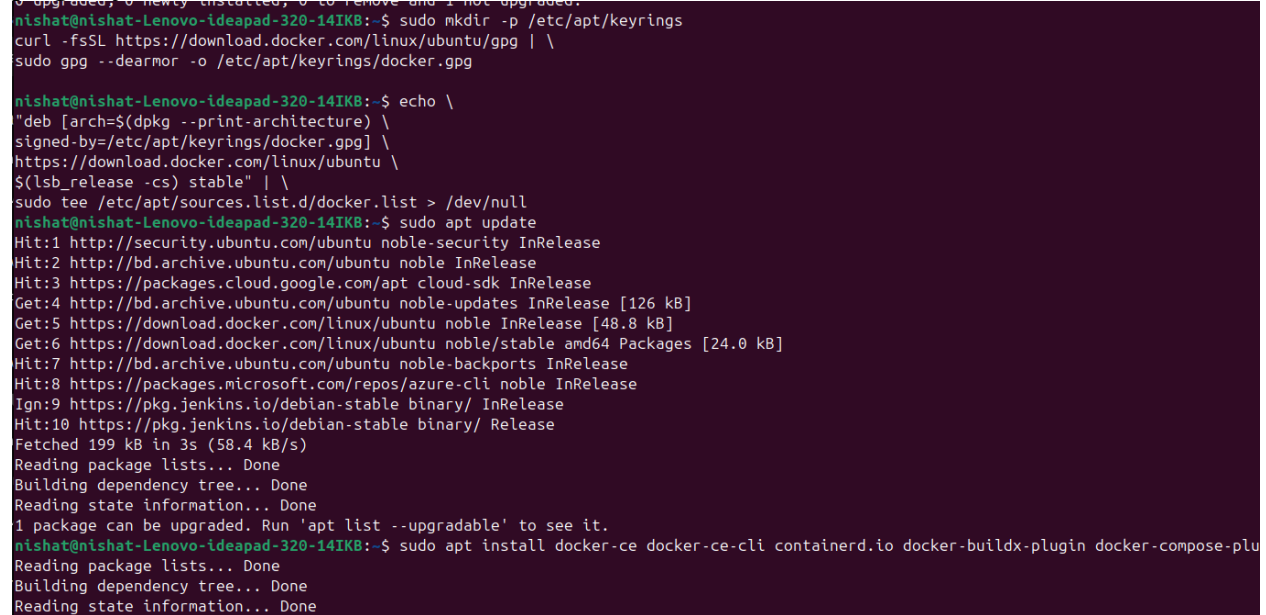
`echo \`

`"deb [arch=$(dpkg --print-architecture) \`

```
signed-by=/etc/apt/keyrings/docker.gpg] \  
https://download.docker.com/linux/ubuntu \  
$(lsb_release -cs) stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

****Install Docker packages:****

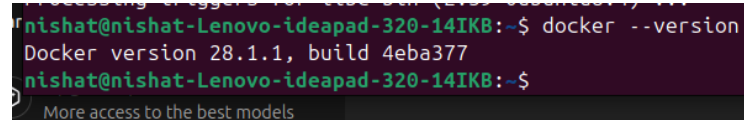
```
sudo apt update  
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-  
plugin -y
```



```
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ sudo mkdir -p /etc/apt/keyrings  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \  
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg  
  
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ echo \  
"deb [arch=$(dpkg --print-architecture) \  
signed-by=/etc/apt/keyrings/docker.gpg] \  
https://download.docker.com/linux/ubuntu \  
$(lsb_release -cs) stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null  
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ sudo apt update  
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease  
Hit:2 http://bd.archive.ubuntu.com/ubuntu noble InRelease  
Hit:3 https://packages.cloud.google.com/apt cloud-sdk InRelease  
Get:4 http://bd.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]  
Get:5 https://download.docker.com/linux/ubuntu noble InRelease [48.8 kB]  
Get:6 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [24.0 kB]  
Hit:7 http://bd.archive.ubuntu.com/ubuntu noble-backports InRelease  
Hit:8 https://packages.microsoft.com/repos/azure-cli noble InRelease  
Ign:9 https://pkg.jenkins.io/debian-stable binary/ InRelease  
Hit:10 https://pkg.jenkins.io/debian-stable binary/ Release  
Fetched 199 kB in 3s (58.4 kB/s)  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
1 package can be upgraded. Run 'apt list --upgradable' to see it.  
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plu  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done
```

****Verify installation:****

```
docker --version
```



```
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ docker --version  
Docker version 28.1.1, build 4eba377  
nishat@nishat-Lenovo-ideapad-320-14IKB:~$
```

****Add user permission:****

```
sudo usermod -aG docker $USER
```

after running the command Now you must log out from your os and log back in, or reboot your system to apply the change.

Logging out and back in is important, otherwise the new permissions won't apply!

****After re-login, test it again:****

After you have completed the installation, you can verify the installation is successful by running the hello-world image.

```
docker run hello-world
```

```
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
e6590344b1a5: Pull complete
Digest: sha256:c41088499908a59aae84b0a49c70e86f4731e588a737f1637e73c8c09d995654
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

ref urls: <https://docs.docker.com/engine/install/ubuntu/>
https://www.tutorialspoint.com/docker/installing_docker_on_linux.htm

Containerizing React App

here i containerized a simple ReactJS Hello World application using Docker, ensuring the app can be built, deployed, and accessed via a Docker container.

****Installed Docker on Ubuntu****

https://dev.azure.com/fronturetech/Fronture%20ERP/_wiki/wikis/Fronture-ERP.wiki/331/Docker-installation-on-ubuntu

****Created a Dockerfile****

[Dockerfile - Repos]https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/reactjs-hello-world/Dockerfile&version=GBDocker&_a=contents

```

GNU nano 2.2
# Building a React app
FROM node:18-alpine AS build

WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .
RUN npm run build

# Serve with NGINX
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

```

****Created .dockerignore****

To reduce build context size, a .dockerignore file was added

```

nishat@nishat-Lenovo-ideapad-320-14IKB: ~/Desktop/react-hello-world/reactjs-hello-world$ cat .dockerignore
node_modules
build
.dockerignore
Dockerfile
nishat@nishat-Lenovo-ideapad-320-14IKB: ~/Desktop/react-hello-world/reactjs-hello-world$

```

****Built Docker Image****

```

ishat@nishat-Lenovo-ideapad-320-14IKB:~/Desktop/react-hello-world/reactjs-hello-world$ docker build -t react-hello-app .
+ ] Building 367.8s (14/14) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 303B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load metadata for docker.io/library/node:18-alpine

```

`docker build -t react-hello-app .`

****Ran the Docker Container in local****

Started a container from the built image and mapped port 3000 of the host to port 80 of the container:

`docker run -d -p 3000:80 react-hello-app`

Verified that the ReactJS app is accessible via:

http://localhost:3000

****Outcomes****

- Successfully containerized the ReactJS application.
- Application served using NGINX inside a Docker container.

**create a snapshot image**

```
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ docker run -d -p 3000:80 --name react-snap-app react-hello-app
75960f4adf3b2547113119dc3759a5ca7ebca9492646f2693f48feb342757ef1
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
75960f4adf3b   react-hello-app "/docker-entrypoint..." 18 seconds ago Up 15 seconds 0.0.0.0:3000->80
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ docker commit react-snap-app react-hello-snapshot:v1
sha256:7927a01b034db40515d96770e864c7e8149c65fd96c90c8163c1ac2ab8bb7b43
nishat@nishat-Lenovo-ideapad-320-14IKB:~$ docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
react-hello-snapshot v1           7927a01b034d     9 seconds ago   49.4MB
```

here i gave this container a name react-snap-app

docker commit ****react-snap-app**** react-hello-snapshot:v1

This means:

You're taking a snapshot of the container named react-snap-app

Creating a new image named react-hello-snapshot with tag v1

Containerizing Laravel App

****Create Dockerfile****

Navigate to the root directory of your Laravel application. Then create a Docker file with out any extension `Dockerfile` and a apache virtual host configuration file like `laravel-world.conf`

Open Dockerfile file :

```
FROM php:8.1-apache
USER root
```

```
RUN apt-get update && apt-get upgrade -y && apt-get install -y \
    procps \
    nano \
    git \
    unzip \
    libicu-dev \
    zlib1g-dev \
```

```
libxml2 \
libxml2-dev \
libreadline-dev \
supervisor \
cron \
sudo \
libzip-dev \
&& docker-php-ext-configure pdo_mysql --with-pdo-mysql=mysqlnd \
&& docker-php-ext-configure intl \
&& docker-php-ext-install \
    pdo_mysql \
    sockets \
    intl \
    opcache \
    zip \
&& rm -rf /tmp/* \
&& rm -rf /var/lib/apt/* \
&& rm -rf /var/lib/apt/lists/* \
&& apt-get clean
```

```
RUN a2dissite 000-default.conf
COPY laraveldock.conf /etc/apache2/sites-available/laraveldock.conf
RUN a2ensite laraveldock.conf
RUN a2enmod rewrite
RUN chown -R www-data:www-data /var/www/html
RUN curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --
filename=composer
WORKDIR /var/www/html
COPY . .
RUN chown -R www-data:www-data /var/www/html/storage
RUN chmod -R 775 /var/www/html/storage
RUN composer install
USER root
```

...

Open laravel-world.conf file:

...

```
<VirtualHost *:80>
ServerName docker23.local
DocumentRoot /var/www/html/public
<Directory /var/www/html/public>
    Options Indexes FollowSymLinks
    AllowOverride All
```

```
    Require all granted
</Directory>
ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```
```

### **\*\*Build and Run\*\***

To build the image:

```
`sudo docker build -t laravel-world:t1`
```

here laravel-world is the image name and t1 is tag.

Run the image:

```
`sudo docker run -d -p 8097:80 --name laravel-container laravel-world:t1`
```

here, specified a port 8097 on localhost, laravel container is the container name and the image we want to run.

### **\*\*Push the Image to Docker hub\*\***

1. Tag Docker Image:

```
`docker tag laravel-world:t1 shakibesaib/laravel-world:t1`
```

2. Login to Docker Hub:

```
`docker login`
```

Provide login credential of Docker hub account

3. Push the Image to Docker Hub:

```
`docker push shakibesaib/laravel-world:t1`
```

### **\*\*Deploy the container in GCP VM\*\***

1. Create a vm in gcp. Ref:

[https://dev.azure.com/fronturetech/Fronture%20ERP/\\_wiki/wikis/Fronture-ERP.wiki/225/Create-VM-Using-GCP-portal\(ubuntu\).](https://dev.azure.com/fronturetech/Fronture%20ERP/_wiki/wikis/Fronture-ERP.wiki/225/Create-VM-Using-GCP-portal(ubuntu).)

2. Connect to SSH

VM instances [CREATE INSTANCE](#) [IMPORT VM](#) [REFRESH](#) [LEARN](#)

[INSTANCES](#) [OBSERVABILITY](#) [INSTANCE SCHEDULES](#)

### VM instances

**Filter** Enter property name or value ? ☰

| <input type="checkbox"/> | Status | Name ↑                        | Zone          | Recommendations | In use by | Internal IP                        | External IP                                                          | Connect                           |
|--------------------------|--------|-------------------------------|---------------|-----------------|-----------|------------------------------------|----------------------------------------------------------------------|-----------------------------------|
| <input type="checkbox"/> | ✓      | <a href="#">laravel-world</a> | asia-south1-c |                 |           | 10.160.0.13 <a href="#">(nic0)</a> | <a href="#">34.93.91.48</a> <a href="#">(nic0)</a> <a href="#">🔗</a> | SSH <span>▼</span> <span>⋮</span> |

#### Related actions ⤴ HIDE

**Explore Backup and DR** NEW  
 Back up your VMs and set up disaster recovery

**Monitor VMs**  
 View outlier VMs across metrics like CPU and network

**Explore VM logs**  
 View, search, analyse and download VM instance logs

**Set up firewall rules**  
 Control traffic to and from a VM instance

**Patch management**  
 Schedule patch updates and view patch compliance on VM instances

**Load balance between VMs** [🔗](#)  
 Set up load balancing for your applications as your traffic and users grow

3. Install docker on vm.<https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-on-ubuntu-20-04>

4. Pull the docker image from Docker Hub  
``sudo docker pull shakibesaib/laravel-world:t1``

5. Login to Docker Hub.

6. Run the docker image.  
``sudo docker run -d -p 80:80 --name laravel-container shakibesaib/laravel-world:t1``

7. Check container.  
``Docker ps``

8. Hit the external ip address and the application would be running.

VM instances [CREATE INSTANCE](#) [IMPORT VM](#) [REFRESH](#) [LEARN](#)

[INSTANCES](#) [OBSERVABILITY](#) [INSTANCE SCHEDULES](#)

### VM instances

**Filter** Enter property name or value ? ☰

| <input type="checkbox"/> | Status | Name ↑                        | Zone          | Recommendations | In use by | Internal IP                        | External IP                                                          | Connect                           |
|--------------------------|--------|-------------------------------|---------------|-----------------|-----------|------------------------------------|----------------------------------------------------------------------|-----------------------------------|
| <input type="checkbox"/> | ✓      | <a href="#">laravel-world</a> | asia-south1-c |                 |           | 10.160.0.13 <a href="#">(nic0)</a> | <a href="#">34.93.91.48</a> <a href="#">(nic0)</a> <a href="#">🔗</a> | SSH <span>▼</span> <span>⋮</span> |

#### Related actions ⤴ HIDE

**Explore Backup and DR** NEW  
 Back up your VMs and set up disaster recovery

**Monitor VMs**  
 View outlier VMs across metrics like CPU and network

**Explore VM logs**  
 View, search, analyse and download VM instance logs

**Set up firewall rules**  
 Control traffic to and from a VM instance

**Patch management**  
 Schedule patch updates and view patch compliance on VM instances

**Load balance between VMs** [🔗](#)  
 Set up load balancing for your applications as your traffic and users grow



### Troubleshoot:

#### 1. **\*\*403 Forbidden\*\***

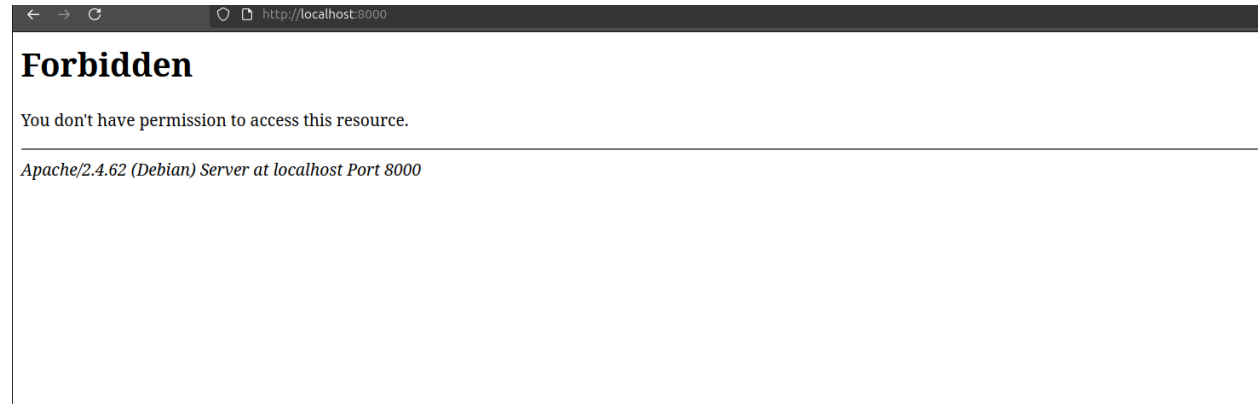
- . Make sure www-data (apache) has enough permission within the docker container to read write and execute.
- . In host configuration make sure document root is perfectly set. ``/var/www/html/public``.

#### 2. **\*\*500 Internal server error\*\***

- . Make The .env file is available at the document root.

3. While building the docker image, make sure all the directories are defined perfectly and all the permission has given with in the docker file.

### ## **\*\*403 Forbidden Error\*\***



Modify Apache to use Laravel's public/ directory

add the following line right after COPY `./var/www/html` in your Dockerfile:

### Update Apache to serve Laravel's public directory

```
>command: RUN sed -i 's|DocumentRoot /var/www/html|DocumentRoot /var/www/html/public|g' /etc/apache2/sites-available/000-default.conf
```

```

GNU nano 7.2 Dockerfile
FROM php:8.2-apache

Install system dependencies
RUN apt-get update && apt-get install -y \
 libonig-dev \
 libzip-dev \
 unzip \
 zip \
 curl \
 git \
 && docker-php-ext-install pdo pdo_mysql zip \
 && a2enmod rewrite

Set working directory
WORKDIR /var/www/html

Copy only the composer files first for caching purposes
COPY composer.json composer.lock /var/www/html/

Install Composer (this is more efficient as it directly pulls the Composer binary from the official Composer image)
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
RUN composer install --no-autoloader --no-scripts

Copy the rest of the app code
COPY . /var/www/html

Update Apache to serve Laravel's public directory
RUN sed -i 's|DocumentRoot /var/www/html|DocumentRoot /var/www/html/public|g' /etc/apache2/sites-available/000-default.conf

Set proper permissions
RUN chown -R www-data:www-data /var/www/html \
 && chmod -R 755 /var/www/html

RUN git config --global --add safe.directory /var/www/html

Install Composer autoload and scripts after copying the app
RUN composer dump-autoloader --optimize

Expose Apache port
EXPOSE 80

```

Rebuild Docker image:

>docker build -t laravel-app .

if old container exist then remove it before running again:

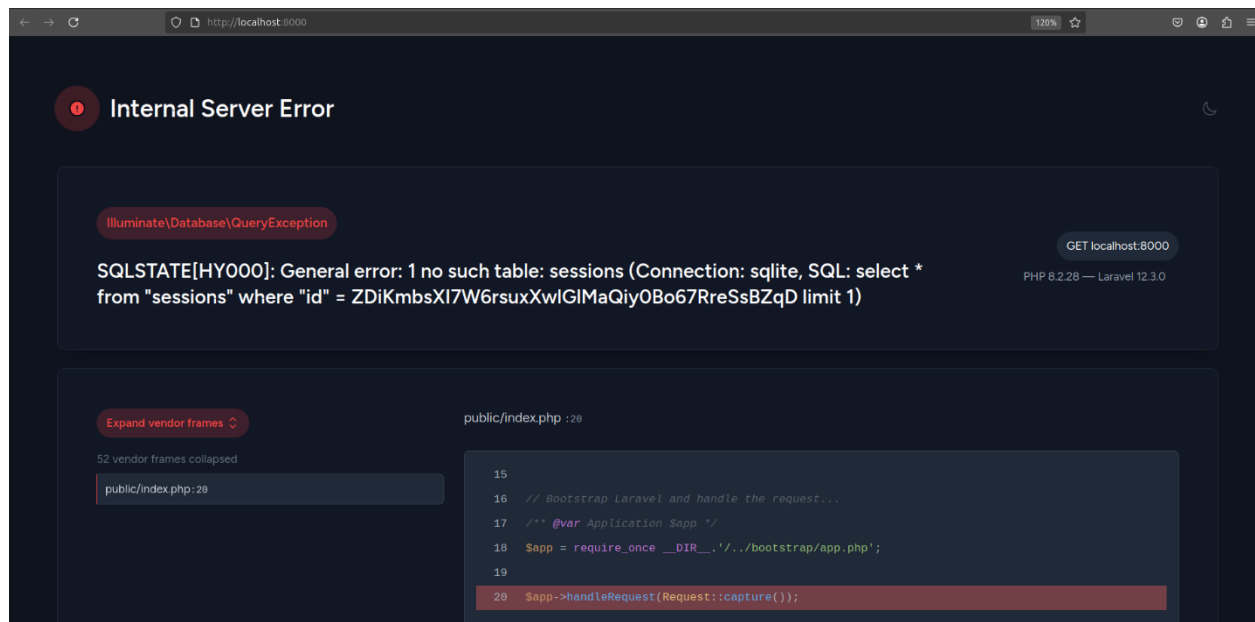
>docker rm -f laravel-container

Run container again:

>docker run -d -p 8000:80 --name laravel-container laravel-app

## \*\*"no such table: sessions" error:

\*\*"no such table: sessions" error means \*\*Laravel is trying to use the `sessions` table from a database\*\* (probably SQLite), but that table doesn't exist yet.



for the sessions table run this :

```
>php artisan migrate
```

You need to run migrations inside the container. Here's how:

Get inside the container:

```
>docker exec -it laravel-container bash
```

Inside the container, run:

```
>php artisan migrate
```

Then exit

Now reload <http://localhost:8000> — the error should be gone.

## Docker compose:

practical experience in containerizing frontend (ReactJS) and backend (Laravel) applications using Docker, and managing them together using Docker Compose.

ReactJS Dockerization:

[https://dev.azure.com/fronturetech/Fronture%20ERP/\\_wiki/wikis/Fronture-ERP.wiki/333/Dockerizing-a-ReactJS-Application](https://dev.azure.com/fronturetech/Fronture%20ERP/_wiki/wikis/Fronture-ERP.wiki/333/Dockerizing-a-ReactJS-Application)

Laravel Dockerization:

[https://dev.azure.com/fronturetech/Fronture%20ERP/\\_wiki/wikis/Fronture-ERP.wiki/244/Dockerization-a-Laravel-App](https://dev.azure.com/fronturetech/Fronture%20ERP/_wiki/wikis/Fronture-ERP.wiki/244/Dockerization-a-Laravel-App)

## Docker Compose Setup:

Created `docker-compose.yml`

[https://dev.azure.com/fronturetech/Fronture%20ERP/\\_git/hands-on-lab-nishat?path=/docker-compose.yml&version=GBDocker&\\_a=contents](https://dev.azure.com/fronturetech/Fronture%20ERP/_git/hands-on-lab-nishat?path=/docker-compose.yml&version=GBDocker&_a=contents)

**\*\*now build the compose\*\***

```
nishat@nishat-Lenovo-ideapad-320-14IKB:~/Desktop/react-hello-world$ docker-compose up --build -d
Creating network "react-hello-world_app-network" with the default driver
Building backend
[+] Building 14.9s (20/20) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.27kB
=> [internal] load metadata for docker.io/library/composer:latest
=> [internal] load metadata for docker.io/library/php:8.2-apache
=> [auth] library/composer:pull token for registry-1.docker.io
=> [auth] library/php:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [stage-0 1/11] FROM docker.io/library/php:8.2-apache@sha256:e2408924aac97ed8dce0ba54adff30443fe7a940a87c
=> [internal] load build context
=> => transferring context: 759.84kB
=> FROM docker.io/library/composer:latest@sha256:d9d52c36baea592479eb7e024d4c1afaba9bdea27d67566c588d290a31b
=> CACHED [stage-0 2/11] RUN apt-get update && apt-get install -y libonig-dev libzip-dev unzip
=> CACHED [stage-0 3/11] WORKDIR /var/www/html
=> CACHED [stage-0 4/11] COPY composer.json composer.lock /var/www/html/
=> CACHED [stage-0 5/11] COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
=> CACHED [stage-0 6/11] RUN composer install --no-autoloader --no-scripts
=> CACHED [stage-0 7/11] COPY . /var/www/html
=> CACHED [stage-0 8/11] RUN sed -i 's|DocumentRoot /var/www/html|DocumentRoot /var/www/html/public|g' /etc
=> CACHED [stage-0 9/11] RUN chown -R www-data:www-data /var/www/html && chmod -R 755 /var/www/html
=> CACHED [stage-0 10/11] RUN git config --global --add safe.directory /var/www/html
=> CACHED [stage-0 11/11] RUN composer dump-autoload --optimize
=> exporting to image
=> => exporting layers
=> => writing image sha256:73e92e657f77217f3d29cc0c6d069259fc17a0f5904866028a42a25094565801
=> => naming to docker.io/library/react-hello-world_backend
Building frontend
[+] Building 11.2s (16/16) FINISHED
=> [internal] load build definition from Dockerfile
```

### Final Verification

After running `docker-compose up -d`, you confirmed:

\* React app accessible at: `http://localhost:3000`

\* Laravel backend accessible at: `http://localhost:8000` [can use other ports as well]

**\*\*ContainerConfig Error:\*\***

The error `KeyError: 'ContainerConfig'` indicates that Docker is trying to access container configuration data that doesn't exist in the expected format.

```
f3f657794ff1_react-hello-world_frontend_1 /docker-entrypoint.sh nginx ... Exit 0
nishat@nishat-Lenovo-ideapad-320-14IKB:~/Desktop/react-hello-world$ docker-compose up -d
Recreating 98d96f8fbd1e_react-hello-world_backend_1 ...
Recreating f3f657794ff1_react-hello-world_frontend_1 ...
ERROR: for 98d96f8fbd1e_react-hello-world_backend_1 'ContainerConfig'
ERROR: for f3f657794ff1_react-hello-world_frontend_1 'ContainerConfig'
ERROR: for backend 'ContainerConfig'
ERROR: for frontend 'ContainerConfig'
Traceback (most recent call last):
 File "/usr/bin/docker-compose", line 33, in <module>
 sys.exit(load_entry_point('docker-compose==1.29.2', 'console_scripts', 'docker-compose')())
```

Here are some steps to troubleshoot and fix this issue:

First, try removing the existing containers:

```

nishat@nishat-Lenovo-ideapad-320-14IKB:~/Desktop/react-hello-world$ docker-compose down
Removing 98d96f8b1e_react-hello-world_backend_1 ... done
Removing f3f657794ff1_react-hello-world_frontend_1 ... done
Removing network react-hello-world_app-network

```

Then rebuild your services again. t will successfully build the composer again.

```

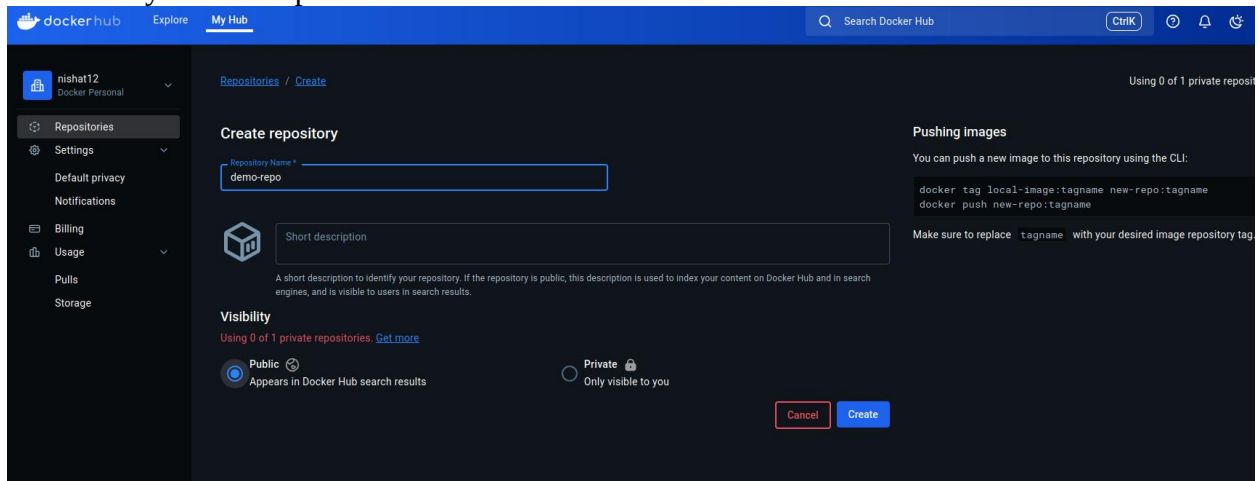
nishat@nishat-Lenovo-ideapad-320-14IKB:~/Desktop/react-hello-world$ docker-compose up --build
Creating network "react-hello-world_app-network" with the default driver
Building backend
[+] Building 14.9s (20/20) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.27kB
=> [internal] load metadata for docker.io/library/composer:latest
=> [internal] load metadata for docker.io/library/php:8.2-apache
=> [auth] library/composer:pull token for registry-1.docker.io
=> [auth] library/php:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [stage-0 1/11] FROM docker.io/library/php:8.2-apache@sha256:e2408924aac97ed8dce0ba54adff3f
=> [internal] load build context
=> => transferring context: 759.84kB
=> FROM docker.io/library/composer:latest@sha256:d9d52c36baea592479eb7e024d4c1afaba9bdea27d67f
=> CACHED [stage-0 2/11] RUN apt-get update && apt-get install -y libonig-dev libzip
=> CACHED [stage-0 3/11] WORKDIR /var/www/html
=> CACHED [stage-0 4/11] COPY composer.json composer.lock /var/www/html/
=> CACHED [stage-0 5/11] COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
=> CACHED [stage-0 6/11] RUN composer install --no-autoloader --no-scripts
=> CACHED [stage-0 7/11] COPY . /var/www/html
=> CACHED [stage-0 8/11] RUN sed -i 's|DocumentRoot /var/www/html|DocumentRoot /var/www/html|
=> CACHED [stage-0 9/11] RUN chown -R www-data:www-data /var/www/html && chmod -R 755 /v
=> CACHED [stage-0 10/11] RUN git config --global --add safe.directory /var/www/html
=> CACHED [stage-0 11/11] RUN composer dump-autoload --optimize
=> exporting to image
=> => exporting layers
=> => writing image sha256:73e92e657f77217f3d29cc0c6d069259fc17a0f5904866028a42a25094565801
=> => naming to docker.io/library/react-hello-world_backend
Building frontend
[+] Building 11.2s (16/16) FINISHED
=> [internal] load build definition from Dockerfile

```

## **\*\*Pushing the React App to Docker Hub\*\***

### ### 1. \*\*Login to Docker Hub\*\*

First, log in to your Docker Hub account  
docker login  
make sure you have repo or create new one.

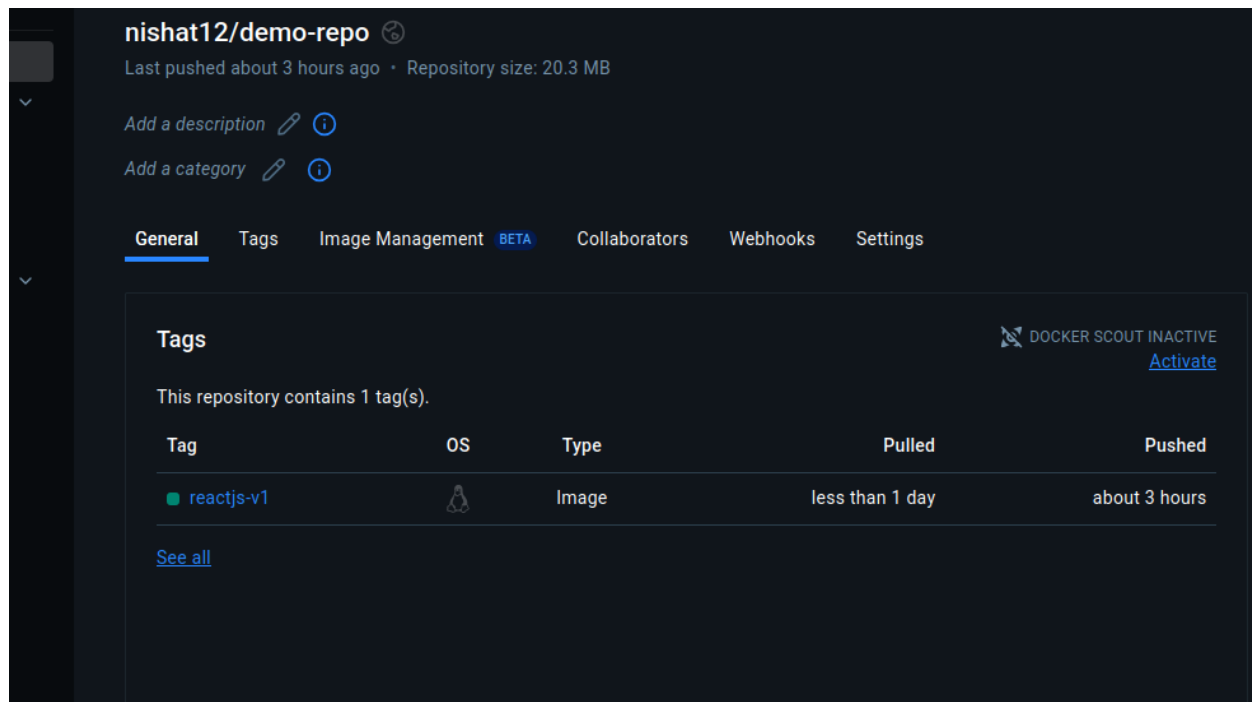


### **\*\*2. Tag Your Docker Image and push it\*\***

To push your image to Docker Hub, you need to tag it with your Docker Hub username and a version. Let's use reactjs-v1 for this example.  
docker tag react-hello-app your-dockerhub-username/your-repo-name:reactjs-v1

```
denied: requested access to the resource is denied
nishat@nishat-Lenovo-ideapad-320-14IK8:~/Desktop/react-hello-world/reactjs-hello-world$ docker tag react-hello-app nishat12/demo-repo:reactjs-v1
nishat@nishat-Lenovo-ideapad-320-14IK8:~/Desktop/react-hello-world/reactjs-hello-world$ docker push nishat12/demo-repo:reactjs-v1
The push refers to repository [docker.io/nishat12/demo-repo]
2790968d63a9: Pushed
0d853d50b128: Pushed
947e805a4ac7: Pushed
811a4dbbf4a5: Pushed
b8d7d1d22634: Pushed
e244aa659f61: Pushed
c56f134d3805: Pushed
d71eae0084c1: Pushed
08000c18d16d: Pushed
reactjs-v1: digest: sha256:f31443b465b8f0d516deba7440bc32bc34a85e2ba01d89c2a75164920d712850 size: 2199
nishat@nishat-Lenovo-ideapad-320-14IK8:~/Desktop/react-hello-world/reactjs-hello-world$
```

Docker will upload your image to your Docker Hub account.



Here's what you need to do on your GCP VM:

1. SSH into your GCP VM

From your local laptop, open a terminal and connect to your VM:

```
gcloud compute ssh nishat-test
```

(Or use the GCP console "SSH" button if you prefer GUI.)

2. Install Docker on your GCP VM

If you haven't installed Docker there yet, install it:

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker
```

Check if Docker installed successfully:

```
nishatafla21@nishat-test:~$ docker version
Client:
 Version: 20.10.24+dfsg1
 API version: 1.41
 Go version: go1.19.8
 Git commit: 297e128
 Built: Sat Oct 12 15:19:49 2024
 OS/Arch: linux/amd64
 Context: default
 Experimental: true

Server:
```

```
docker --version
```

### 3. Login to Docker Hub on GCP VM

On your VM terminal:

```
docker login
```

Enter your Docker Hub username and password.

### 4. Pull your Docker image

Pull your React app image from Docker Hub:

```
docker pull nishat21/demo-repo:reactjs-v1
```

### 5. Run the container

Run the container and map port 80 inside the container to port 8080 on the VM:

```
docker run -d -p 8080:80 your-dockerhub-username/react-hello-app:v1
```

**## \*\* ⚡ Quick Important Note:\*\***

if you encounter this problem make sure to allow firewall

```
nishatafla21@nishat-test:~$ docker run -d -p 80:8080 nishat12/demo-repo:reactjs-v1
856e81cd291635c1c7b77f589039993145b8902f83c058888b85288b1578d130
docker: Error response from daemon: driver failed programming external connectivity on endpoint elastic_moser (d87d7756e358d6579c196e935e229e7ec4af9f3f9494343bab37332610d1dcfe): Bind for 0.0.0.0:80 failed: port is already allocated.
nishatafla21@nishat-test:~$ docker run -d -p 8080:80 nishat12/demo-repo:reactjs-v1
```

If you cannot access the app, you must allow TCP port 8080(or, other ports) in the GCP Firewall:

Go to GCP Console → VPC Network → Firewall rules → Create Rule.

Allow Ingress traffic on TCP port 8080 for your VM.

|                          |                    |         |             |            |        |       |      |         |    |
|--------------------------|--------------------|---------|-------------|------------|--------|-------|------|---------|----|
| <input type="checkbox"/> | default-allow-http | Ingress | http-server | IP ranges: | tcp:80 | Allow | 1000 | default | On |
|--------------------------|--------------------|---------|-------------|------------|--------|-------|------|---------|----|

### 6. Access the app in your browser

Now you need to get the External IP address of your GCP VM:

Go to GCP Console → VM instances → Find nishat-test → Copy the External IP.

Then, open this URL from your laptop browser:

```
nishatafla21@nishat-test:~$ docker run -d -p 8080:80 nishat12/demo-repo:reactjs-v1
472aad38483424e0c6037d827a97087d090cec33e81a891bcffdbdd72552afb7
nishatafla21@nishat-test:~$ docker version
```

http://<your-vm-external-ip>:8080

You should see your ReactJS app!